

Codeforces Round 1074 (Div. 4)

A. Perfect Root

1 second, 256 megabytes

A positive integer x is a perfect root if there exists an integer y such that $\sqrt{y} = x$. For example, 5 is a perfect root because $\sqrt{25} = 5$.
For each test case, output n distinct perfect roots. Note that the values only need to be distinct within each test case; you can use the same value in different test cases.

Input

The first line of the input contains a single integer t ($1 \leq t \leq 20$) — the number of test cases.

The only line of each test case contains an integer n ($1 \leq n \leq 20$) — the number of perfect roots to output.

Output

For each test case, output n distinct perfect roots. Each perfect root x must be in the range $1 \leq x \leq 10^9$.

input
3 1 2 5
output
1 2 4 2 102 43 1 21

For the first test case:

- 1 is a perfect root because $\sqrt{1} = 1$.

For the second test case:

- 2 is a perfect root because $\sqrt{4} = 2$.
- 4 is a perfect root because $\sqrt{16} = 4$.

B. Prefix Max

1.5 seconds, 256 megabytes

You are given an array of n integers $a_1, a_2, \dots, a_n$.

The value of an array is the sum of the maximums of each prefix of the array. More formally, the value of an array a is $\sum_{i=1}^n \max(a_1, \dots, a_i)$. For example, the value of the array $[1, 2, 1]$ is $\max(1) + \max(1, 2) + \max(1, 2, 1) = 1 + 2 + 2 = 5$.

You can choose two indices i and j and swap elements a_i and a_j ; this operation can be applied **at most** one time.

Find the maximum possible value of the array a after at most one operation.

Input

The first line of the input contains a single integer t ($1 \leq t \leq 100$) — the number of test cases.

The first line of each test case contains a single integer n ($2 \leq n \leq 50$) — the length of the array a .

The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^4$) — the array a .

Output

For each test case, output the maximum possible value of the array a after the swap has been performed.

input
4 5 2 1 4 5 3 2 5 1 3 3 2 1 2 6 7
output
25 10 9 14

For the first test case, we can swap a_1 with a_4 to get the array $[5, 1, 4, 2, 3]$, which has a value of $\max(5) + \max(5, 1) + \max(5, 1, 4) + \max(5, 1, 4, 2) + \max(5, 1, 4, 2, 3)$.

For the second test case, the current value of the array is $\max(5) + \max(5, 1) = 10$. If we were to swap a_1 and a_2 , a would be equal to $[1, 5]$, which has a value of $\max(1) + \max(1, 5) = 6$, meaning the best option is to not perform a swap.

C. Shifted MEX

2 seconds, 256 megabytes

You are given an array of n integers $a_1, a_2, \dots, a_n$. You are allowed to perform the following operation once.

- Select an integer x (which may be negative), and for each value i ($1 \leq i \leq n$), set $a_i = a_i + x$.

For example, if $a = [1, 3, 4, 2]$, and you perform the operation with $x = 3$, a is now equal to $[4, 6, 7, 5]$.

Output the maximum possible value of $\text{MEX}(a)^*$ after the operation is performed.

* $\text{MEX}(a)$ is defined as the smallest non-negative integer that is not present in the array. For example, $\text{MEX}([1, 2, 0, 5])$ is 3, and $\text{MEX}([1, 2, 4, 9])$ is 0.

Input

The first line of the input contains a single integer t ($1 \leq t \leq 1000$) — the number of test cases.

The first line of each test case contains a single integer n ($1 \leq n \leq 3000$) — the length of array a .

The second line contains n integers $a_1, a_2, \dots, a_n$ ($-10^9 \leq a_i \leq 10^9$) — the array a .

It is guaranteed that the sum of n over all test cases does not exceed 3000.

Output

For each test case, output the maximum possible value of $\text{MEX}(a)$ after the operation has been performed.

input
6 1 4 5 0 1 1 2 3 2 1 1 4 4 2 3 6 5 2 4 1 0 -1 6 -1 1 2 3 5 6

output
1 4 1 3 4 3

For the first test case, performing the operation with $x = -4$ makes $a = [0]$, and $\text{MEX}([0]) = 1$.

For the second test case, the **MEX** is already 4, which is the highest possible, so we can perform the operation with $x = 0$, which will not change the array.

D. OutOfMemoryError

2 seconds, 256 megabytes

Bessie has an array of n integers $a_1, a_2, \dots, a_n$. Bessie performs m operations on the array. The i -th operation sets $a_{b_i} = a_{b_i} + c_i$.

Unfortunately, due to rising RAM costs, Bessie's computer has limited memory, and whenever any element of the array is greater than h , her computer crashes, and every element in the array is reset to its original value.

After all operations have been performed, output the array a .

Input

The first line of the input contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains three integers n, m, h ($1 \leq n, m \leq 2 \cdot 10^5, 1 \leq h \leq 10^9$) — the length of array a , the number of operations performed, and the maximum value that Bessie's computer can store without crashing.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq h$) — the array a .

The next m lines contain two integers b_i, c_i ($1 \leq b_i \leq n, 0 \leq c_i \leq 10^9$) — the operations that Bessie performs on the array.

It is guaranteed that the sum of n and the sum of m over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output the array a after all operations have been performed.

input
3 3 4 5 1 2 1 1 4 2 4 3 3 2 0 5 3 1 1 1 1 1 1 1 1 1 1 2 1 4 4 1 1 0 0 0 1 1 4 4 3 3 4 4
output
1 2 4 1 1 1 1 1 1 0 0 0

For the first test case, the array a is changed as follows:

- Before all operations, $a = [1, 2, 1]$.
- After the first operation, $a = [5, 2, 1]$.
- After the second operation, $a = [5, 6, 1]$, but since $6 > h$, the computer crashes, and $a = [1, 2, 1]$.
- After the third operation, $a = [1, 2, 4]$.
- After the fourth operation, $a = [1, 2, 4]$.

For the third test case, each operation causes the computer to crash, so $a = [1, 0, 0, 0]$.

E. The Robotic Rush

3 seconds, 256 megabytes

There exists an infinitely long number line.

On the number line, there are n robots and m spikes, each located at a specific point on the number line. The i -th robot is located at position a_i , and the i -th spike is located at position b_i . If a robot touches a spike, it dies.

k instructions are transmitted to the robots, with each instruction being to either move left by one unit or to move right by one unit.

For each value i ($1 \leq i \leq k$), output how many robots are still alive after the first i instructions have been processed.

Input

The first line of the input contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains three integers n, m, k ($1 \leq n, m, k \leq 2 \cdot 10^5$) — the number of robots, spikes, and instructions respectively.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 10^9$) — the locations of the robots. It is guaranteed that all elements of a are distinct.

The third line contains m integers $b_1, b_2, \dots, b_m$ ($0 \leq b_i \leq 10^9$) — the locations of the spikes. It is guaranteed that all elements of b are distinct.

The fourth line contains a string of length k — the instructions transmitted to the robots. Each character is either **L**, representing an instruction to move to the left, or **R**, representing an instruction to move to the right.

It is guaranteed that the sum of each of n, m, k over all test cases does not exceed $2 \cdot 10^5$.

Additional constraint: it is guaranteed that there are no robots and spikes at the same position.

Output

Output k integers, where the i -th integer indicates how many robots are alive after the first i instructions have been processed.

For Python users, make sure to select PyPy3 / PyPy2 (depending on which version of Python you're using) instead of Python3 or Python2 when submitting.

input
3 2 1 3 0 1 2 LRR 2 3 3 2 4 1 3 5 LRL 3 2 3 1 3 7 9 6 RRL
output
2 2 1 0 0 0 3 2 2

For the first test case:

- The first robot will move to positions $0 \rightarrow -1 \rightarrow 0 \rightarrow 1$, so it will not die.
- The second robot will move to positions $1 \rightarrow 0 \rightarrow 1 \rightarrow 2$, so it will die after processing the third instruction, since there is a spike at position 2.

For the second test case, both robots will die after moving once.

F. BattleCows

2 seconds, 256 megabytes

Farmer John is sending a cow to compete in the International Bovine Olympiad, but he's too lazy to write problems for a contest, so he does the next most reasonable thing: a fighting tournament!

Farmer John has 2^n cows standing in a line, with the i -th cow having a skill level of a_i . Each cow starts in a stack containing only itself, and the skill level of the stack is equal to the XOR-sum* of the skill levels of all the cows in it. For example, if the stack consists of cows 1, 3, 9 from bottom to top, then the skill level of the stack is $1 \oplus 3 \oplus 9 = 11$.

The following process repeats until there is only one stack.

- Every stack in an odd position (1st, 3rd, etc.) starts a fight with the stack to its right. For example, the 1st stack will fight with the 2nd stack, the 3rd stack will fight with the 4th stack, etc.
- The stack with the higher skill level will win the match, with the left-most stack winning in case of a tie.
- The winning stack will jump on top of the losing stack and shift itself over such that there are no gaps left by the defeated stacks.

In order to make this more exciting, Farmer John created q potions, such that the i -th potion sets the skill level of the cow who drinks it to c_i . Farmer John wants to test his potions on the cows, and so he gives the i -th potion to cow b_i and then runs the tournament. For each trial, Farmer John wants to know how many cows are above the cow that was given the potion in the final stack.

Farmer John's potions wear off quickly, so when the tournament ends, the cow that was given the potion will **have its skill level return to what it originally was**. In other words, all queries are independent.

*The XOR-sum of an array $x_1, x_2, \dots, x_y$ is equal to $x_1 \oplus x_2 \oplus x_3 \dots x_{y-1} \oplus x_y$ where $\oplus$ denotes the bitwise XOR operation.

Input

The first line of the input contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains two integers n and q ($1 \leq n \leq 18, 1 \leq q \leq 2 \cdot 10^5$) — where 2^n is equal to the number of cows in the tournament and q is the number of potions.

The second line contains 2^n integers $a_1, a_2, \dots, a_{2^n}$ ($1 \leq a_i \leq 2^{30}$) — the skill levels of the cows.

The next q lines contain two integers b_i and c_i ($1 \leq b_i \leq 2^n, 1 \leq c_i \leq 2^{30}$) — the index of the cow that is given the potion and the new skill level of the cow.

It is guaranteed that the sum of 2^n over all test cases does not exceed $2^{18} = 262144$ and that the sum of q over all test cases does not exceed $2 \cdot 10^5$.

Output

For each trial, output how many cows are above the cow that was given the potion in the final line.

input
4
2 2
1 3 5 7
1 1
4 8
1 2
1 3
1 4
1 2
3 4
1 8 3 10 2 5 7 1
5 3
8 12
1 9
2 1
2 1
1 2 3 4
3 1

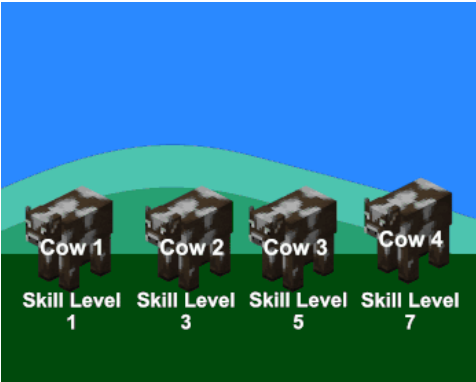
output
1
0
0
1
5
0
2
3
1

For the first round of the first test case, the skill levels of the cows don't change, and the matches go as follows:

- Cows 1 and 2 fight. Since cow 2's skill level is higher than cow 1's skill level, cow 2 wins the fight, so its stack has cows 1, 2 in that order, and the stack's skill level is now $3 \oplus 1 = 2$.
- Cows 3 and 4 fight. Since cow 4's skill level is higher than cow 3's skill level, cow 4 wins the fight, so its stack has cows 3, 4 in that order, and the stack's skill level is now $7 \oplus 5 = 2$.
- The two remaining stacks fight. Since the first stack's skill level is equal to the second stack's skill level, the first stack wins the fight, since its index is lower than the second stack's, so the final stack has cows 3, 4, 1, 2 in that order, and its skill level is now $2 \oplus 2 = 0$.

Since we gave the potion to cow 1, our answer is 1, since there is one cow above cow 1.

A visualization of the round is shown below.



For the second round of the first test case, cow 4's skill level is now 8, and the matches go as follows:

- Cows 1 and 2 fight. Since cow 2's skill level is higher than cow 1's skill level, cow 2 wins the fight, so its stack has cows 1, 2 in that order, and the stack's skill level is now $3 \oplus 1 = 2$.
- Cows 3 and 4 fight. Since cow 4's skill level is higher than cow 3's skill level, cow 4 wins the fight, so its stack has cows 3, 4 in that order, and the stack's skill level is now $8 \oplus 5 = 13$.
- The two remaining stacks fight. Since the second stack's skill level is higher than the first stack's skill level, the second stack wins the fight, so the final stack has cows 1, 2, 3, 4 in that order, and its skill level is now $2 \oplus 13 = 15$.

Since we gave the potion to cow 4, our answer is 0, since there are no cows above cow 4.

G. Mixing MEXes

2 seconds, 256 megabytes

You are given n arrays $a_1, a_2, \dots, a_n$.

The following operation is performed exactly once:

- Choose any array among $a_1, a_2, \dots, a_n$. Suppose you've chosen array a_i ($1 \leq i \leq n$).
- Choose any element in array a_i . Suppose you've chosen the j -th element of a_i , denoted by $a_{i,j}$ ($1 \leq j \leq |a_i|$, where $|a_i|$ denotes the length of array a_i).
- Choose any other array among $a_1, a_2, \dots, a_n$ that is not a_i . Suppose you've chosen a_k ($1 \leq k \leq n, k \neq i$).
- Add $a_{i,j}$ to the back of array a_k . Then, remove $a_{i,j}$ from a_i .
- The value of this operation (i, j, k) is defined as the sum of each array's MEX after the operation is performed. More formally, the

value of an operation after the operation is performed is $\sum_{i=1}^n \text{MEX}(a_i)$.

Evaluate the sum of the values of all possible distinct independent operations. Two operations are distinct if the ordered triple of integers (i, j, k) is different.

$\text{MEX}(a)$ is defined as the smallest non-negative integer that is not present in the array. For example, $\text{MEX}([1, 2, 0, 5])$ is 3, and $\text{MEX}([1, 2, 4, 9])$ is 0.

Input

The first line of the input contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains a single integer n ($2 \leq n \leq 2 \cdot 10^5$) — the number of arrays.

The next n lines start with l_i ($1 \leq l_i \leq 10^5$) — the length of the i th array — then contain l_i integers $a_1, a_2, \dots, a_{l_i}$ ($0 \leq a_{i_j} \leq 10^6$) — the array a_i .

It is guaranteed that the sum of l_i over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output the sum of the values of all possible distinct operations.

input
6 2 1 0 2 1 2 3 1 1 2 2 3 3 4 5 6 5 4 1 7 8 10 2 5 6 2 0 7 2 6 6 2 6 8 2 1 3 3 0 1 2 2 6 0 0 1 2 2 3 3 0 2 3 10 1 0 9 7 8 0 1 5 6 4 3 2 8 4 3 8 6 2 5 0 1 7 2 3 0 1 0 4 0 2 3 1 9 2 0 5 4 1 3 0 0 0 7 6 3 2 4 1 8 0 5 3 2 4 1 0 4 0 3 1 1 3 0 3 2
output
6 0 50 8 43 19202

For the first test case, we have 3 possible distinct operations:

- $i = 1, j = 1, k = 2$: The arrays are now [] and [0, 1, 2], which have a mex of 0 and 3 respectively, so the value of the operation is 3.
- $i = 2, j = 1, k = 1$: The arrays are now [0, 1] and [2], which have a mex of 2 and 0 respectively, so the value of the operation is 2.
- $i = 2, j = 2, k = 1$: The arrays are now [0, 2] and [1], which have a mex of 1 and 0 respectively, so the value of the operation is 1.

For the second test case, since no array contains a zero, the value of all operations will be 0.

H. BattleCows 2

3 seconds, 256 megabytes

Farmer John wants to host another tournament with n cows, where the i -th cow has a skill level of a_i . The following process repeats until there is only one cow in the line.

- The first cow in the line fights the second cow in the line, and the cow with the higher skill level wins. If there is a tie, the first cow wins.
- The winning cow's skill level is set to $x + y$, where x is the skill level of the winning cow and y is the skill level of the losing cow.
- The losing cow leaves the line.

However, to maintain accuracy to the real USACOW competition, a cow is able to cheat up to k times. This means that even if it loses the match, Farmer John will treat it as if the losing cow won the match, meaning that the losing cow's skill level will be set to $x + y$, where x is the skill level of the winning cow and y is the skill level of the losing cow, and the winning cow will leave the line.

A position x is good for a cow i if cow i can be removed from its original position and inserted at index x **without changing the order of the other cows** and be the only cow remaining in the line once the tournament has ended, assuming no other cow cheats.

For each cow in the line, calculate the number of good positions for that cow.

Input

The first line of the input contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains two integers n and k ($2 \leq n \leq 2 \cdot 10^5, 0 \leq k < n$) — the number of cows and the number of cheats a cow can use.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — the skill levels of the cows.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output n integers, where the i -th integer denotes the number of good positions for cow i .

input
7 2 0 2 1 2 0 1 1 3 1 1 1 3 3 0 2 1 1 3 1 1 3 1 4 1 1 2 1 3 7 2 1 3 3 17 39 3 12
output
2 0 1 1 2 2 3 2 0 0 3 3 2 4 4 4 4 4 6 6 7 7 5 7

For the first test case, cow 2 would lose no matter what it was positioned, and cow 1 would win no matter where it was positioned.

For the second test case, both cows could win at position 1, but could not win at position 2, meaning that they each have 1 good position.

For the third test case, let's look at cow 1.

- If cow 1 were at position 1, then it would win its first match, making its skill level equal to 2, and could cheat to win its second match.
- If cow 1 were at position 2, then it would have to cheat to win its first match, but would then lose its second match.
- If cow 1 were at position 3, then it would have to cheat to win its first match, and would have no more matches.

Hints: Codeforces Round 1074 (Div. 4)

Taken from the official editorial, not written by us. Read one hint at a time.

A. Perfect Root (official hints)

Hint 1: Fix an integer x , how could you find out the value of y ?

Hint 2: Square both sides of the equation, for what values of x is x^2 an integer?

B. Prefix Max (official hints)

Hint 1: If we were to change the problem such that you could perform as many swaps as you wanted, the answer would remain the same.

Hint 2: Once our running sum encounters the maximum element of the array, the max of that prefix will always be the maximum element of the array.

C. Shifted MEX (official hints)

Solution 1: Hint 1: Duplicate values don't matter, nor does the order of the elements.

Solution 1: Hint 2: Let's consider the inverse problem, where we have a array that is equal to $[0, 1, 2, \dots, k - 1]$ and we can add x to each value in the array. No matter which value of x we choose, the values in a are consecutive.

Solution 2: Hint 1: In order for the MEX to be greater than 0, 0 has to be contained in the array.

Solution 2: Hint 2: Since there are at most 3000 elements in a , there are 3000 possible values of x that have a MEX greater than 0.

D. OutOfMemoryError (official hints)

Hint 1: Clearly, resetting all of the values is too slow, but we don't have to update all the elements at the same time.

Hint 2: If an element hasn't been modified after the last crash, it's value in the final array is the same as its value in the original array.

E. The Robotic Rush (official hints)

Hint 1: Each robot's movements is independent of another robot's movements.

Hint 2: We only need to care about the spikes directly to the right and left of the robot, if they exist.

Hint 3: To figure out when a robot dies, we don't need to care about the position of the robot, only the distance to its left and right spikes.

Hint 4: We only care about the left-most a robot has been and the right-most a robot has been.

F. BattleCows (official hints)

Hint 1: A single cow will only face n matches.

Hint 2: XOR is cumulative, meaning the order of the cows in a stack doesn't change its skill level.

Hint 3: Similarly, the order of the cows in the stack doesn't change the final position of the cow who was given the potion in the final line.

Hint 4: You can calculate range XOR queries in $O(1)$ using prefix sum.

G. Mixing MEXes (official hints)

Hint 1: The only time adding an element changes the MEX of an array is if the added element is equal to the MEX of the array.

Hint 2: For a given element, the element is removed $n - 1$ times, and the element is added to another array $n - 1$ times.

Hint 3: We can treat removing an element from the array and adding it to another array as two separate interactions.

H. BattleCows 2 (official hints)

Hint 1: I had originally proposed this problem with a subtask to solve for Cow 0 only, try that.

Hint 2: We can simulate each cow as if it started as position 0.

Hint 3: A cow only needs cheat at most $\log(\max(a_i))$ times, since its skill level is (at least) double after each cheat.

Hint 4: Prefix sums, binary search, and helper methods are your friend.

Source: <https://codeforces.com/blog/entry/150288>

[Codeforces](https://codeforces.com/) (c) Copyright 2010-2026 Mike Mirzayanov
The only programming contests Web 2.0 platform